



UNIVERSITÀ
DEGLI STUDI
FIRENZE

DINFO

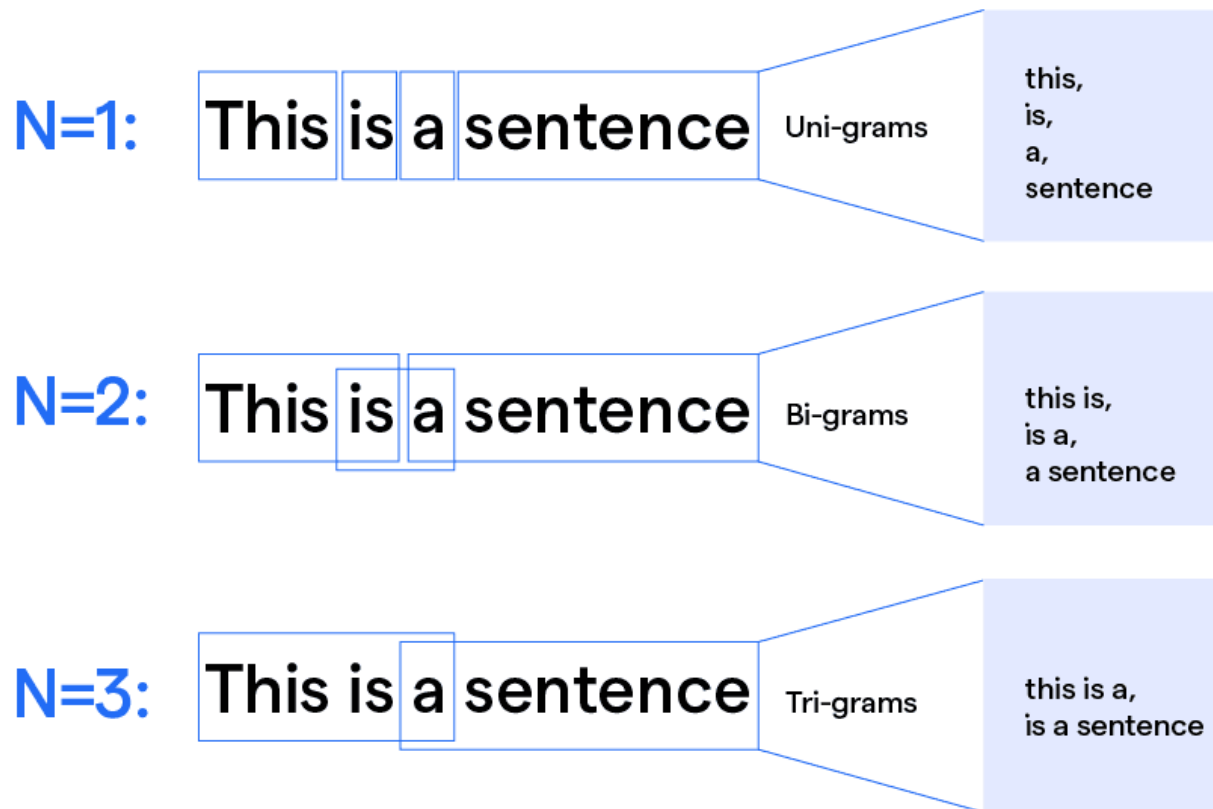
Dipartimento di
Ingegneria dell'Informazione



OpenMP Parallelization of N-gram Histograms (Bigrams & Trigrams)

Chiara Gori

- Definition: contiguous sequence of n items (characters or words) extracted from a text



- Goal:
 - Compute frequency histograms of bigrams and trigrams
 - both at character and word level
 - Parallelize workflow and show benefits
- Real-world use cases:
 - Text mining
 - Subword tokenization (LLMs)
 - Linguistic analysis
 - ...
- Test corpus: *Jane Eyre* (Project Gutenberg)
 - Replicated to obtain corpora of ~200MB and ~700MB

- Sliding window over text file
 - Requires iterating over millions of positions in the text
- Updating shared data structures
- Two variants:
 - Char n-grams:
 - Uniform workload per iteration + direct array access
 - Word n-grams:
 - Variable workload per iteration + hash map use

→ explore different parallelization strategies

- Preprocessing
 - Text cleaning → 37-symbol alphabet (26 letters + 10 digits + underscore), all lowercase, spaces/punctuation turned into underscores
 - Char n-grams span word boundaries
- Char n-grams:
 - Base 37 direct addressing → direct index into a flat vector
→ $O(1)$ + zero collisions + no hashing
- Word n-grams:
 - Text split in words on underscores
 - Windows of consecutive words become keys in map

- Example:
 - Bigram 'he' at position $p = 1$ in the string 'the_cat':
 - $i = 0: c = 'h' \rightarrow \text{offset} = 'h' - 'a' + 10 = 17 \rightarrow \text{index} = 0 \times 37 + 17 = 17$
 - $i = 1: c = 'e' \rightarrow \text{offset} = 'e' - 'a' + 10 = 14 \rightarrow \text{index} = 17 \times 37 + 14 = 643$
- histogram[643]++

- Per-thread local histogram + sequential final merge
- Char n-grams:
 - Uniform workload → static scheduling policy
- Word n-grams:
 - 3 scheduling policies
 - Static
 - Dynamic
 - Guided
 - 4 chunk sizes:
 - Auto
 - 64
 - 512
 - 4096

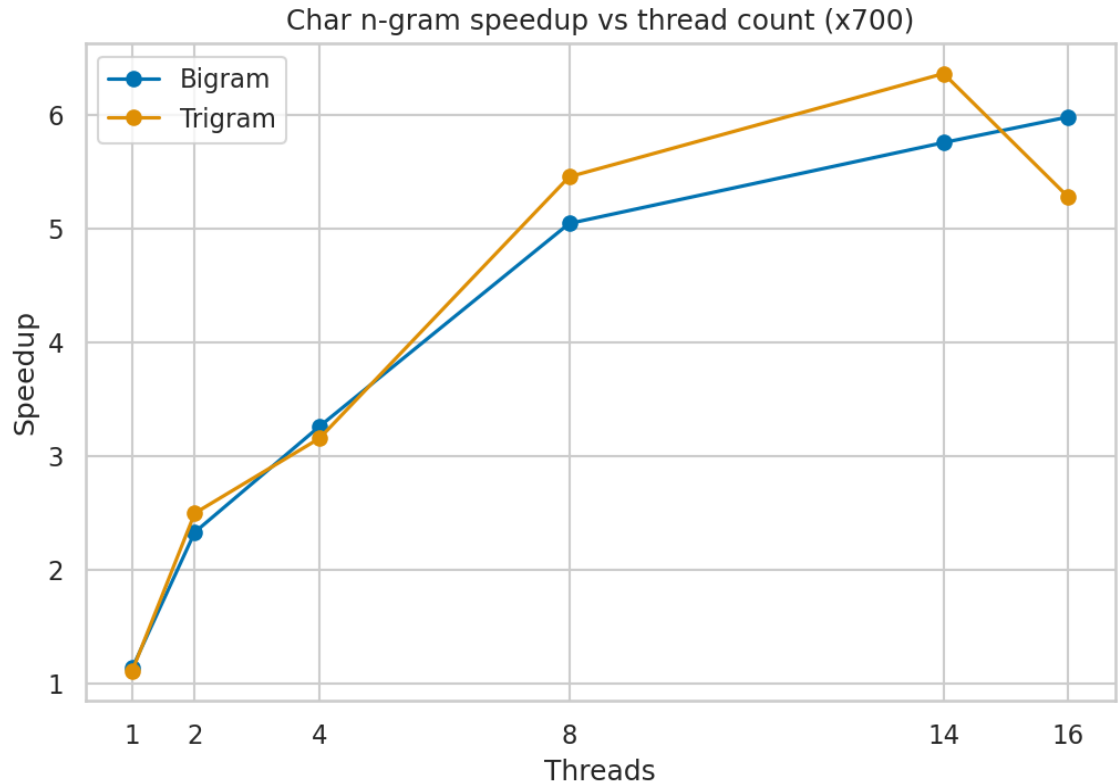


- Entire file read into memory
 - reduces I/O overhead
 - Correctness validation
 - Char n-grams:
 - merge = simple element-wise sum of small, fixed-size vectors → cheap
 - Word n-grams:
 - merge = iterating over unordered_maps → much more expensive
 - (string comparisons, dynamic allocations)
- asymmetry in merge cost

- HW:
 - Intel Core Ultra 7 155U, 7 physical cores / 14 logical threads, 12GB RAM (SW: WSL2, Ubuntu 22.04)
- Thread counts tested: 1, 2, 4, 8, 14, 16
- Methodology:
 - 7 runs per configuration: first 2 discarded (warm-up) + average over 5 runs
- Main metric:
speedup = sequential time / parallel time (wall-clock)

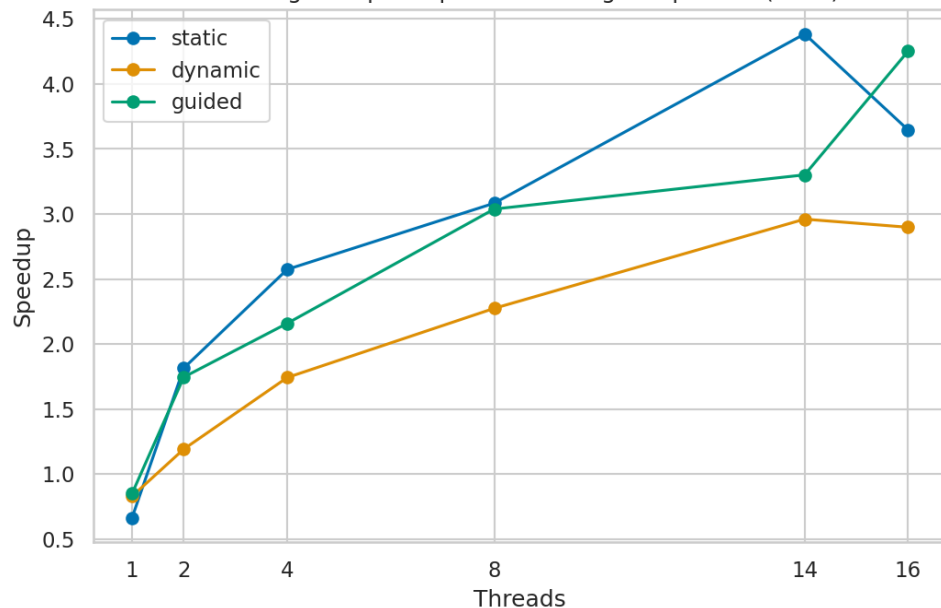
Speedup: Character n-grams

- x700 corpus
- Good scalability until 14 threads
- 16 threads: oversubscription

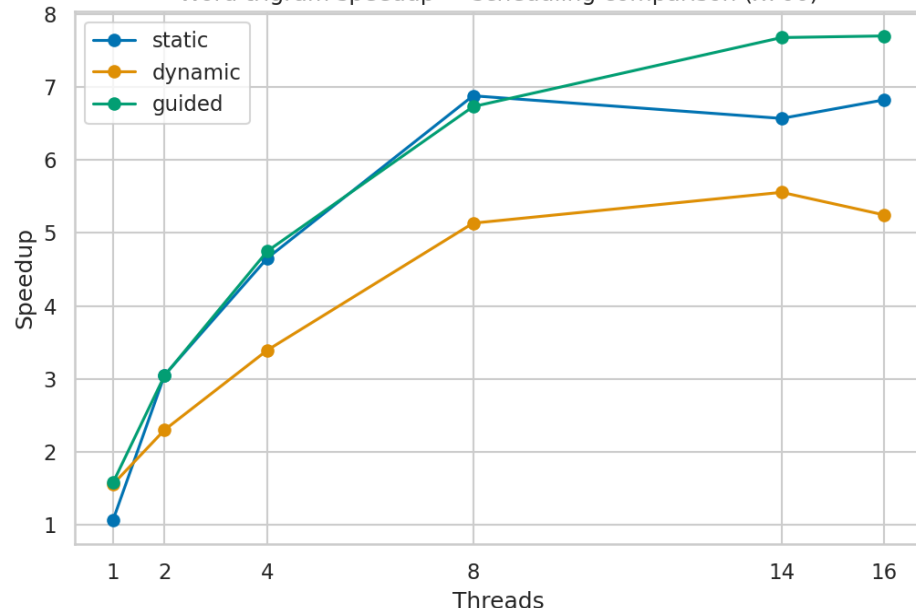


Speedup: Word n-grams

Word bigram speedup — scheduling comparison (x700)

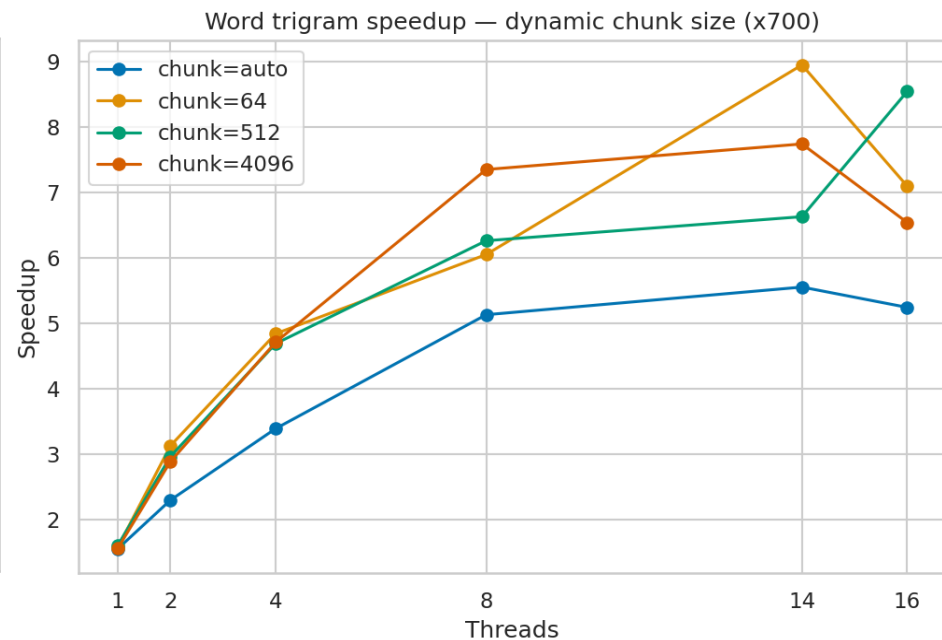
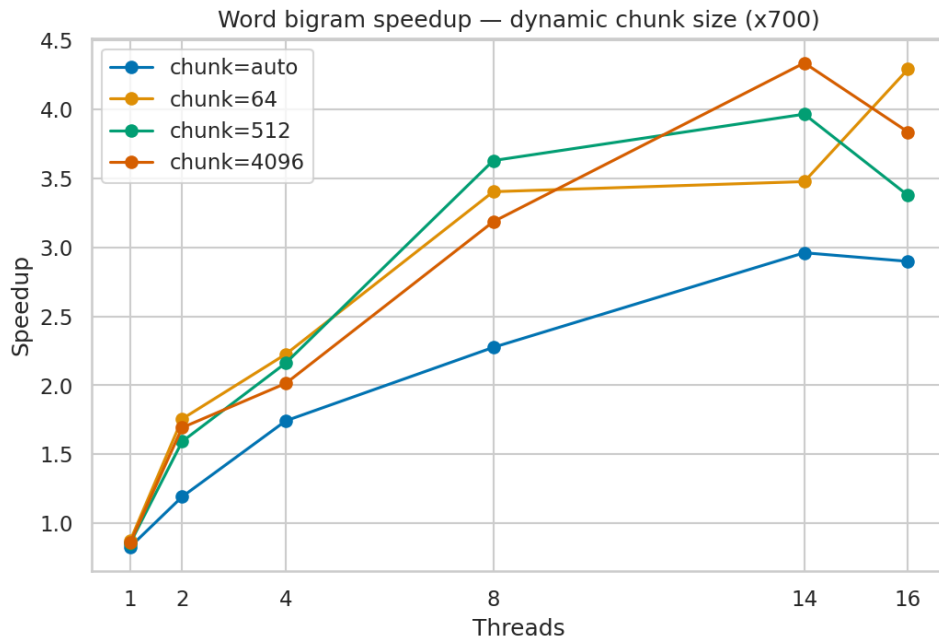


Word trigram speedup — scheduling comparison (x700)



- x700 corpus
- Chunk size = auto
- Static and guided scheduling ✓
- Dynamic ✗ (scheduling overhead)

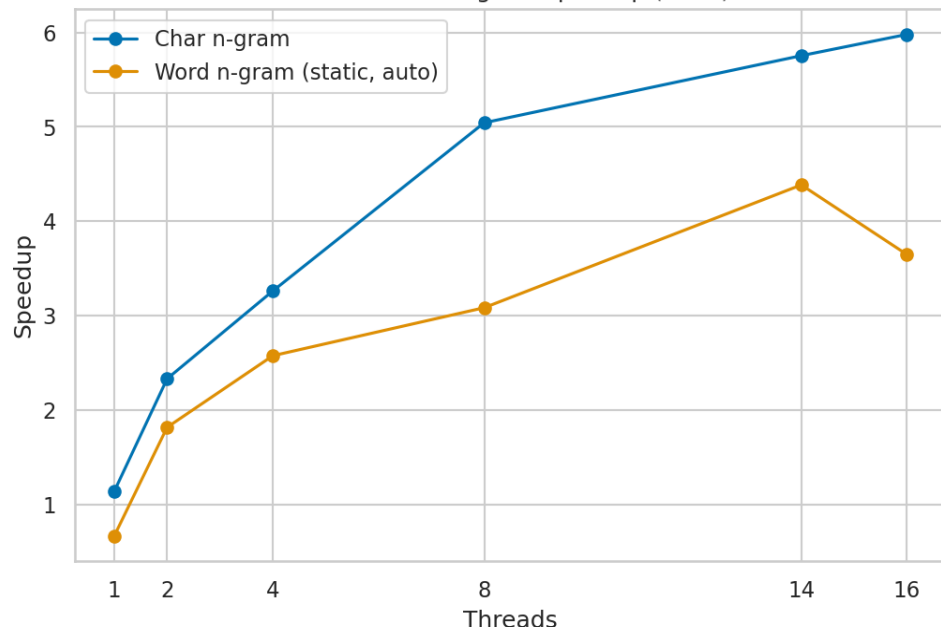
Speedup: Word n-grams



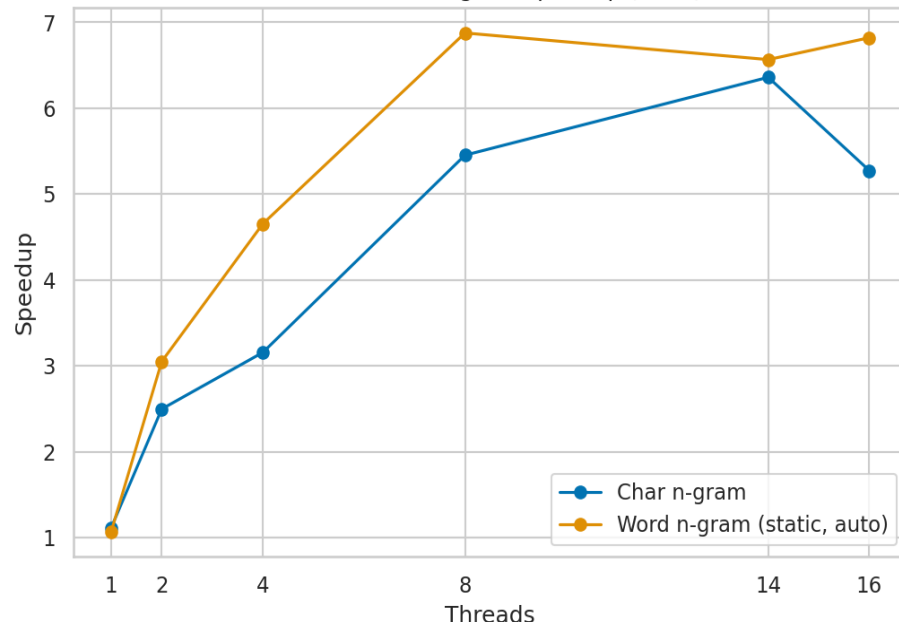
- Different chunk sizes tested
- Dynamic scheduling + chunk size = 64 + 14 threads
→ Best result overall (8.95x speedup)

Char vs Word n-grams

Char vs Word bigram speedup (x700)



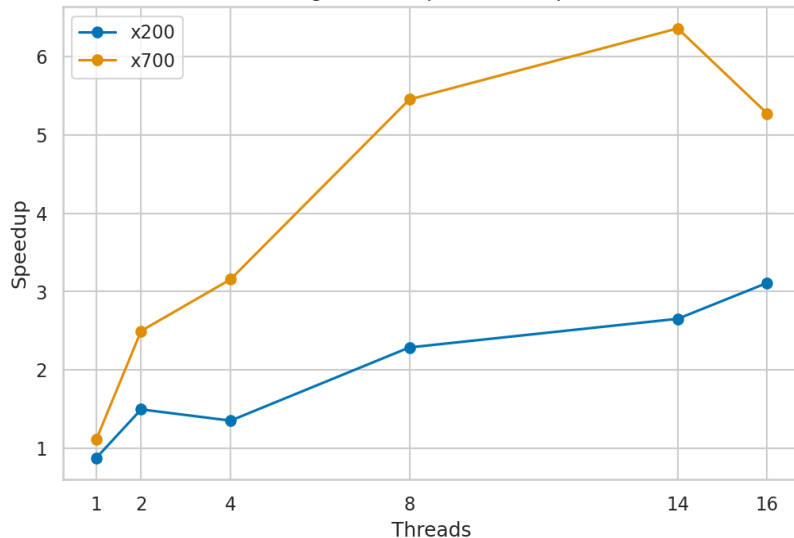
Char vs Word trigram speedup (x700)



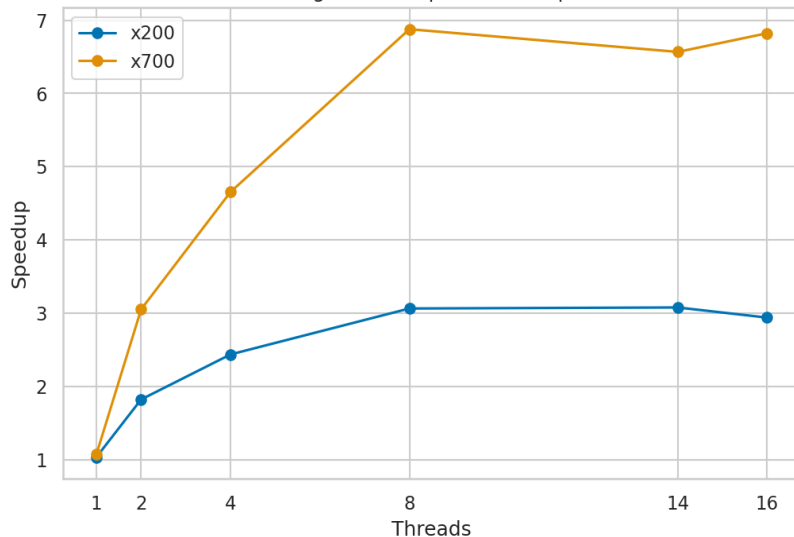
- Char bigrams > word bigrams
 - Uniform workload + lighter final merge
 - Word trigrams > char trigrams
 - Per iteration-cost is higher → parallel computation dominates merge overhead → better scalability
- parallelization pays off more when the ratio of computation to synchronization overhead is higher

Effect of Corpus Size

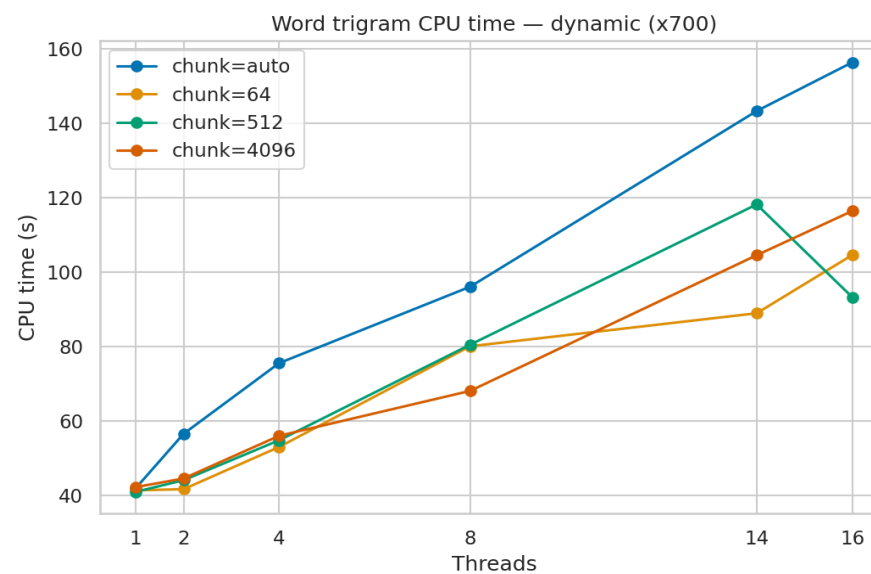
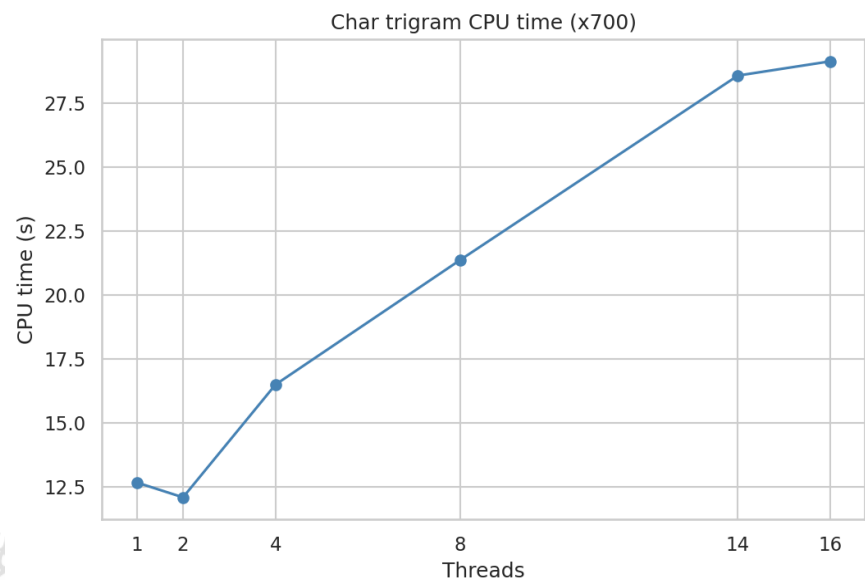
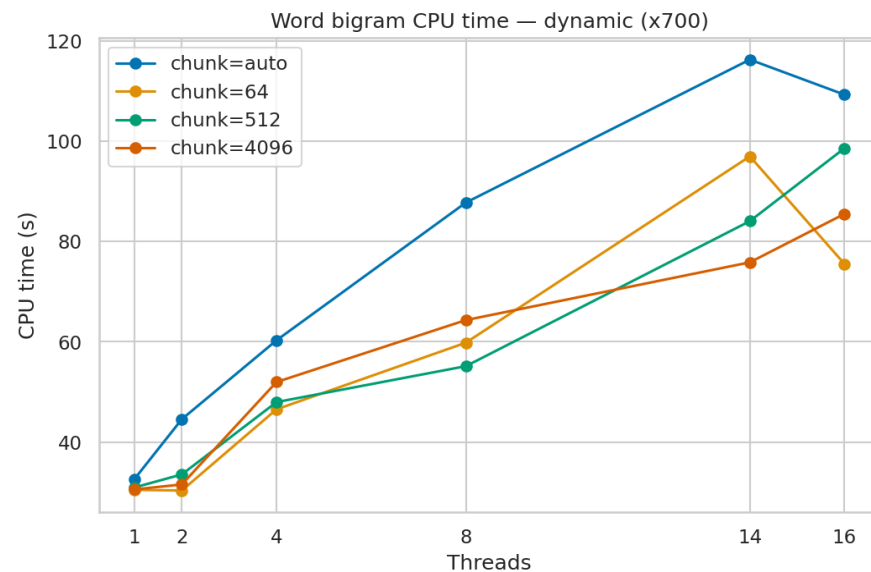
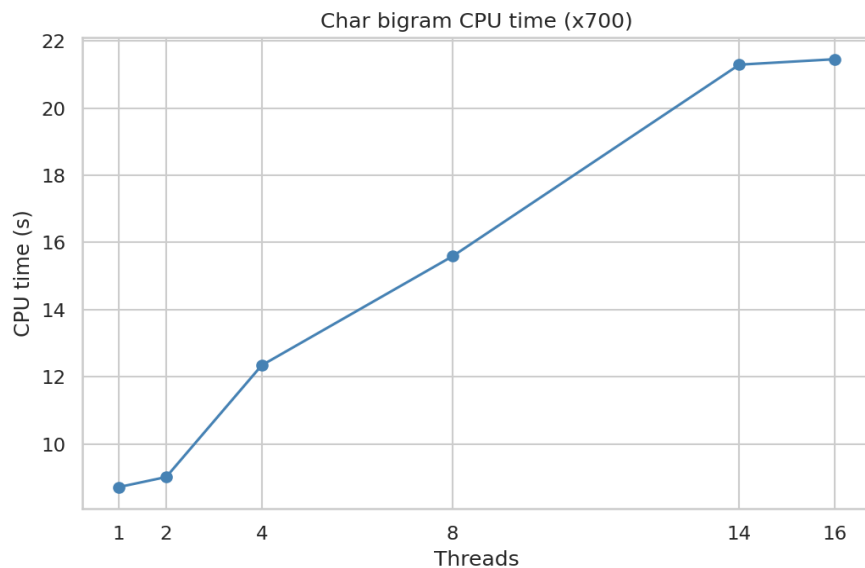
Char Trigram — corpus size comparison



Word Trigram — corpus size comparison

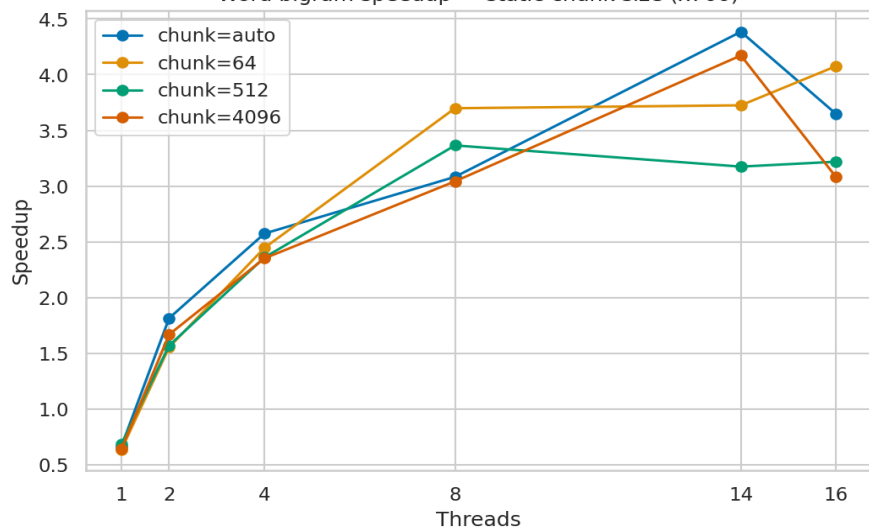


- x700 consistently scales better than x200
- Large workloads amortize parallelization overhead
- Coherent with Amdahl's law:
 - Impact of sequential part is reduced for larger workloads

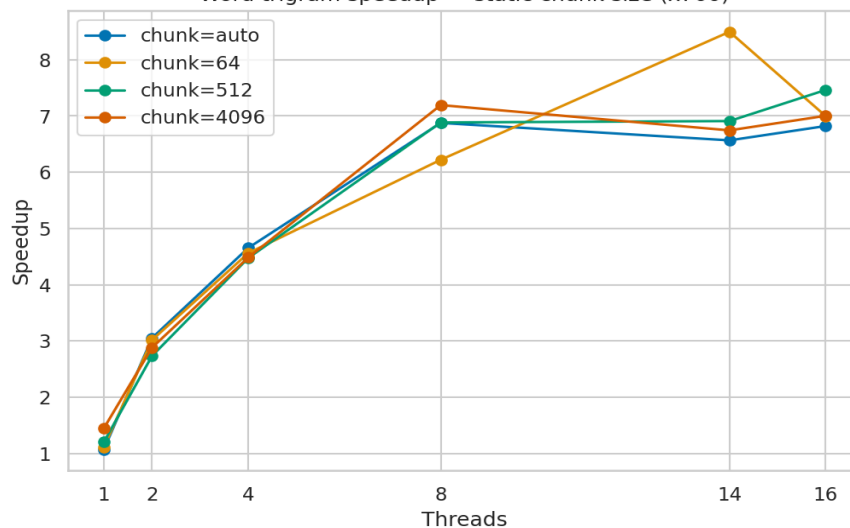


- Optimal configuration depends on the nature of the problem:
 - Char n-grams: uniform workload
→ static scheduling
 - Word n-grams: requires tuning of scheduling and chunk size
→ dynamic scheduling + chunk size = 64
→ achieves higher speedups
- 14 threads (= logical threads) is the best configuration in almost every case
- Corpus size comparison confirmed the behaviour expected from Amdahl's law
- Scheduling overhead can be more costly than the load-balancing effect it provides

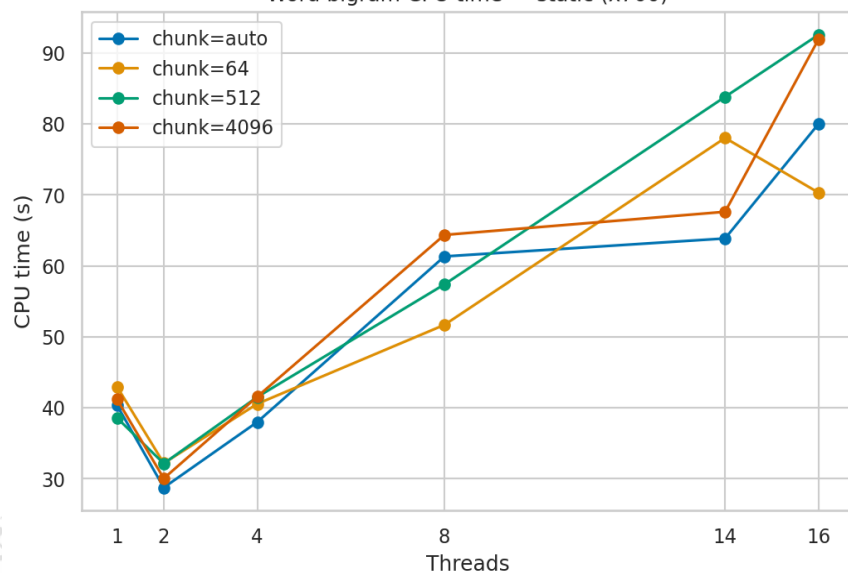
Word bigram speedup — static chunk size (x700)



Word trigram speedup — static chunk size (x700)



Word bigram CPU time — static (x700)



Word trigram CPU time — static (x700)

